

Zoos Compute Test Engineer

Job-Prep Deep-Dive — Doing the Work, Not Passing the Interview

May 2026

Contents

1	How to Use This Guide	2
2	The Mission and the Platform	4
2.1	What Zoos's compute platform actually is	4
2.2	The autonomy data path (so you know what you're protecting)	4
2.3	Where you sit in the org	4
3	The Four Phases of Manufacturing Test	5
3.1	The four phases	5
3.2	Why the phase matters: the 10x rule	5
3.3	What lives at each phase	5
3.4	The test-coverage allocation skill	6
4	Design Verification vs Manufacturing Test	7
4.1	The two modes side by side	7
4.2	Shmoo, margining, go/no-go	7
4.3	The unifying idea: capture the parameter, not just the verdict	7
5	PCIe, At the Register Level	9
5.1	Config space is just a file you can read	9
5.2	AER: the registers that catch link errors	9
5.3	The write-1-to-clear discipline (your tool's core primitive)	11
5.4	Link speed, width, and reading the LTSSM	12
5.5	Equalization, presets, and why links fall back	13
5.6	Lane Margining at the Receiver (the scope-free eye)	13
5.7	Decoding the AER Header Log (turning a bit into a sentence)	14
5.8	DPC — when the device vanishes on purpose	14
5.9	Self-testing the AER pipeline (aer-inject)	14
5.10	The BERT: measuring bit-error rate to a confidence level	15
5.11	Bringing up a <i>custom</i> PCIe device	15
5.12	PCIe failure-signature triage	16
5.13	Gen5/Gen6, switches, and what's coming	16
6	NVMe Storage, For Test Engineers	18
6.1	The model in one paragraph	18
6.2	SMART / Health — your pass/fail gate	18
6.3	The commands that matter beyond SMART	19
6.4	Stress and data integrity with <code>fio</code>	20
6.5	Firmware update flow (you will own this)	20

6.6	Form factors and therm/hot-swap	20
7	GPUs, For Test Engineers	21
7.1	ECC — the GPU’s error story (analogous to AER)	21
7.2	Thermal-under-load is the test, not a snapshot	21
7.3	DCGM is the manufacturing-grade tool	22
7.4	Multi-GPU and NVLink (awareness)	23
8	Memory (DRAM), For Test Engineers	24
8.1	What you’re testing	24
8.2	EDAC: reading memory errors in Linux	24
8.3	Stressing memory	25
9	High-Speed Links: One Mental Model for PCIe, GMSL, and Ethernet	26
9.1	GMSL — cameras over coax	26
9.2	Automotive Ethernet — one pair, master/slave	27
10	CAN / CAN-FD, For Test Engineers	29
11	Instruments and Measurement	30
11.1	The bench and what each instrument is for	30
11.2	How you talk to them: VISA + SCPI	30
11.3	Measurement discipline (where credibility is won or lost)	30
11.4	What you actually measure on a compute board	31
11.5	The shared-instrument server pattern	31
11.6	Reliability & stress screening (where thermal/vibration belong)	31
12	The Test Station and the Test Framework	33
12.1	Anatomy of a station	33
12.2	Test program architecture	33
12.3	Releasing and versioning test programs	33
13	Test Economics: Deployment, Runtime, and Yield	35
13.1	Yield	35
13.2	Runtime	35
13.3	Deployment	35
14	Mass-Production Realities	36
14.1	Takt, cycle time, and parallel / multi-up test	36
14.2	Test-station architecture and the test executive	36
14.3	Traceability & genealogy	36
14.4	Golden units & cross-site correlation	36
14.5	FAI, CM workflows, MES/OEE	36
14.6	The statistics that turn parameters into limits	37
14.7	False-fail vs escape economics — and why it’s not symmetric here	37
15	The Linux Command-Line Debugging Toolbox	38
16	Your First 90 Days — Technical Ramp	39

1 How to Use This Guide

You got the job. This guide is not interview prep — your 296-page study guide (Zoox_Study_Guide-11.pdf) already did that. This one assumes you’re walking onto the floor and need to *do the work*: design and release manufacturing test solutions for Zoox’s compute platform.

Where the study guide answers “*what is AER?*” for an interviewer, this guide answers “*the link is throwing correctable errors on lane 7 under thermal load — what do I actually do, in what order, with which register?*”

Three rules for reading it:

1. It is organized around your job description, not around topics. Every section maps to a line in the JD: the four manufacturing phases, custom PCIe devices, GPUs/NVMe/memory, high-speed links, instruments, test deployment/runtime/yield.
2. It goes one level deeper than the study guide and says so. When a topic is already well covered in the study guide, I point you there (e.g., “study guide §4.5”) and spend the space here on what it leaves out.
3. It is paired with a toolkit. The companion toolkit/ directory implements a lot of what’s described here — the PCIe BERT engine, AER decode, lane margining, NVMe/GPU/GMSL checks, the pytest harness. Read a section, then go read the code that does it. Concepts you can run beat concepts you can recite.

The one-sentence version of the job. *Build test programs that prove every interface on a compute board works — at speed, under load, at temperature — and catch the bad units before they cost 10× more to find downstream.* Everything in this guide serves that sentence.

A note on what you already bring. Your X-ES PCIe BERT work, the FastAPI station dashboard, the pytest-driven fixtures, the instrument drivers — that is *exactly* this job, on different hardware. The deltas you’re closing in this guide are: (a) the specific silicon (NVIDIA GPUs, NVMe, GMSL deserializers, custom PCIe cards), (b) the manufacturing-phase framing Zoox uses, and (c) going from “ran a BERT” to “own the PCIe diagnostic strategy across PCBA → vehicle.”

2 The Mission and the Platform

2.1 What Zoox’s compute platform actually is

Zoox builds a purpose-built robotaxi — a symmetric, bidirectional vehicle with no steering wheel. The compute platform is the brain: a set of server-grade compute assemblies that ingest sensor data (cameras, lidar, radar), run the perception → prediction → planning → control stack, and command the vehicle. The JD’s phrase “*server-grade computer assemblies, custom PCIe devices, and PC components such as storage and memory*” tells you the shape of it:

- Server-class SoCs/CPU(s) + GPU(s) for the AI/perception workload. Think data-center-grade parts in an automotive enclosure — ECC memory, lots of PCIe lanes, high power and heat in a sealed, vibrating, temperature-cycled box.
- Custom PCIe devices. Zoox designs its own PCIe cards — sensor-interface cards, GMSL aggregators, FPGA/accelerator cards, switch/fan-out boards. “Custom” is the important word: there is no vendor datasheet test plan, no NVIDIA-supplied diagnostic. *You* write the test that proves the board works. This is where a senior test engineer earns their title.
- Storage and memory. NVMe SSDs (OS, logging, map and model storage — sensor logging alone is enormous) and DRAM (DDR4/DDR5, almost certainly ECC).
- High-speed links. PCIe (Gen3/4/5) between everything; GMSL from cameras; automotive Ethernet (radar/lidar/inter-module); CAN/CAN-FD for the vehicle bus. Every one of these is a SerDes link that can train wrong, drift with temperature, or fail a marginal lane — which is most of what you’ll debug.
- Linux. The whole thing runs Linux, and your test stations run Linux. Your command-line debugging fluency is load-bearing.

2.2 The autonomy data path (so you know what you’re protecting)

```
Cameras (e.g. AR0820) --MIPI CSI-2--> GMSL serializer (MAX96717)
      --coax up to 15 m through the harness--> GMSL deserializer (MAX96712)
      --CSI-2 / PCIe--> Compute SoC --PCIe--> GPU(s) + NVMe + NIC
Lidar / Radar --Automotive Ethernet (100/1000BASE-T1)--> Compute
Vehicle bus --CAN / CAN-FD--> Compute
```

Why this matters to you: a single weak PCIe lane, a GMSL link that won’t lock at 85 °C, or an NVMe drive that throttles under sustained logging can silently degrade perception. In a robotaxi that’s a safety issue, not a customer-annoyance issue. The manufacturing test is the gate that keeps a marginal board from ever reaching a vehicle. That’s the weight behind “ship only good units.”

2.3 Where you sit in the org

You’re in the Compute program, on manufacturing test & diagnostics. The JD says you’ll “*collaborate closely with electrical engineering and software teams to develop and validate new products.*” In practice your daily orbit is:

- Electrical Engineering (EE) / hardware design — they design the boards; you test them. You’ll get schematics, board files, and bring-up reports. When your test finds a failure, EE is who you hand the evidence to. The better your evidence (decoded AER, lane margining, eye/thermal correlation), the faster the fix.
- Software / firmware — drivers, BSP, firmware images. Your tests run on their Linux image and flash their firmware. New firmware → you re-qualify the test.
- Manufacturing / NPI / operations — the people who run your tests on the line. Your test has to be fast, robust, and operator-proof.
- Contract Manufacturers (CMs) — external partners who build boards at volume. You release test programs *to them* and support them remotely. (Guide B has a whole section on this — it’s a big part of a senior role.)
- Quality / reliability — yield data, RMA/field returns, corrective action.

You are the person who turns “the hardware team thinks it works” into “we have data proving 1,000 units work, and a gate that stops the ones that don’t.”

3 The Four Phases of Manufacturing Test

This is the backbone of the whole job and a phrase straight out of the JD: “*implement the correct test coverage across all phases of manufacturing (PCBA, module, system, vehicle).*” If you internalize one framework before day one, make it this one. Every test you write lives at one of these phases, and the central skill is putting each test at the earliest phase that can catch its defect.

3.1 The four phases

Phase	What the unit is	Who runs it	What you’re proving
PCBA	Bare PCB + components, just off the SMT line	CM, usually	Built correctly: right parts, good solder, no shorts/opens, powers on, basic boot
Module	PCBA + heatsink + enclosure + connectors = a functional unit	CM or Zoox	Every interface works at speed, under load, at temperature; firmware loaded; calibrated
System	Multiple modules integrated into the compute box/rack	Zoox	Modules talk to each other; full-load power/thermal; system boots and runs the stack
Vehicle	Compute installed in the car with real sensors	Zoox	End-to-end: real cameras lock over GMSL, sensors stream, vehicle-level EOL checks

3.2 Why the phase matters: the 10× rule

The cost to find and fix a defect rises ~10× at each phase you let it escape to:

Caught at	Rough relative cost	Why
PCBA	1×	Rework a single board in the line, automatically
Module	10×	Disassemble enclosure/heatsink, rework, re-test
System	100×	Tear down an integrated system, isolate which module
Vehicle	1000×	Pull compute from a vehicle, diagnose in situ
Field (RMA)	10,000×	Truck roll, downtime, brand/safety risk

This single curve drives test strategy. It’s *why* you “shift left” — push coverage to the earliest phase that can detect a given defect class. A solder short should die at PCBA (ICT), not surface as a PCIe link failure at system test. A marginal Gen4 lane that only fails at 85 °C can’t be caught at room-temperature PCBA test — it needs a thermal stress at module test. Knowing which defects are catchable where is the core of “correct test coverage.”

3.3 What lives at each phase

PCBA test (mostly at the CM, mostly not your code, but you must understand it):

- AOI (Automated Optical Inspection) — cameras check for missing/misplaced/ wrong/tombstoned parts and gross solder defects.
- ICT (In-Circuit Test) — a bed-of-nails fixture probes nets to measure resistance/capacitance, check for shorts/opens, and verify component values.

- Boundary scan / JTAG (IEEE 1149.1) — shifts test patterns through device scan chains to test interconnects (BGA balls you can't probe) and program flash/CPLD.
- Flying probe — ICT without a custom fixture, for low volume/prototypes.
- First power-on / boot — does it come up, draw the right current, reach a prompt. Often programs the initial bootloader/firmware here.

Module test (the heart of *your* job): the unit is now a sealed, functional module with its thermal solution. This is where your interface tests run:

- Enumerate everything: `lspci`, `nvme list`, `nvidia-smi`, NIC/CAN/GMSL presence.
- Prove each link trains at the expected speed and width (PCIe Gen4 x16, NVMe Gen4 x4, etc.).
- Stress + error counting: run traffic and watch AER/SMART/ECC counters — your BERT and diagnostic tools live here.
- Thermal/burn-in: soak at temperature and/or under load; re-check links and errors (catches the marginal-lane-at-temperature defects PCBA can't).
- Power characterization: measure rail voltages/currents with a DMM/scope; verify they're in spec under load.
- Firmware/version verification: flash and confirm firmware/BSP versions.
- Calibration where applicable.

System test: modules are integrated. Now you test the *interactions* — inter-module PCIe/Ethernet links, the system power budget under full load, system-level thermals (fans, airflow, the whole box), and that the integrated system boots and runs. Failures here are more expensive to localize, so module test should have already proven each module in isolation.

Vehicle test (EOL — End Of Line): compute is in the car with real sensors. Cameras must lock over real GMSL harnesses (15 m of coax, real connectors, real EMI), all sensors stream, CAN talks to the vehicle, and the vehicle-level functional checks pass before it ships.

3.4 The test-coverage allocation skill

When EE hands you a new board, the senior-engineer question is: *for each way this can fail, what's the cheapest phase that can catch it, and what test detects it there?* You build a small matrix:

Failure mode	Catchable at	Test
Missing/wrong component	PCBA	AOI / ICT
BGA solder void under the GPU	PCBA (X-ray) + Module (thermal cycling surfaces it)	X-ray; thermal soak + link error monitor
PCIe lane marginal at temperature	Module (not PCBA)	Stress + AER + lane margining at hot/cold
NVMe throttles under sustained write	Module	<code>fio</code> soak + SMART temperature/throttle log
GMSL won't lock over full-length harness	Vehicle (real harness)	Link-lock + frame capture at EOL
Inter-module link marginal	System	System link enumeration + stress

You will not get this matrix perfect on day one — nobody does. But showing up *thinking in this matrix* is what separates a senior hire from a script-runner.

Day-one talking point. “I think about test coverage as a placement problem: for each defect class, find the earliest, cheapest phase that can detect it, and make sure a test lives there. The expensive mistakes are defects that are only *detectable* late but were *introducible* early.” That sentence is the JD's “correct test coverage across all phases” said back in your own words.

4 Design Verification vs Manufacturing Test

The JD asks you to span two modes that share instruments, code, and physics but differ in goal, statistics, and output: “*support the test and validation of prototype designs*” is Design Verification (DV); “*build and release test solutions for the manufacturing lines*” is Manufacturing Test (MT). Conflating them is a classic mistake; fluently moving a test between them is the senior skill. This chapter is the framing; the toolkit is built around it.

One-line framing. DV asks “*how good is this design, and where are its edges?*” (*measure everything, small N, characterize*). MT asks “*is this specific unit good enough, fast?*” (*go/no-go against limits, huge N, capture the few parameters that let you tune those limits later*).

4.1 The two modes side by side

Dimension	Design Verification (DV)	Manufacturing Test (MT)
Question	“How good is the <i>design</i> ? Where are its margins/edges?”	“Is <i>this unit</i> good enough — and fast?”
Output	Characterization data, margin maps, the limits themselves	A go/no-go verdict (+ a few captured parameters)
Sample size	Small N (EVT ~20–50; DVT ~50–500)	Huge N (every unit; PVT ~300–2,000 then full volume)
Method	Characterization, shmoo, margining, corner/stress sweeps	Go/no-go against fixed limits, fast
Conditions	Voltage/temp/frequency corners, worst-case combos	Nominal (+ targeted stress where a defect demands it)
Time budget	Hours–days per unit acceptable	Seconds–minutes per unit (takt-bound)
Run by	Test/EE engineers in the lab	Operators on the line / at the CM
Statistic	Distribution shape, design margin, C_{pk} of the <i>design</i>	FPY, escape/false-fail rates, C_{pk}/P_{pk} against limits

The industry build-phase vocabulary maps onto this: EVT (Engineering Validation, ~20–50 units, “does it meet functional requirements”), DVT (Design Verification, ~50–500 units, “can it be *manufactured* to spec” — heavy characterization/margining), PVT (Production Validation, ~300–2,000 units, “can the *line* hit its metrics”). DV work lives in EVT/DVT; MT is what PVT proves out and mass production runs.

4.2 Shmoo, margining, go/no-go

- A shmoo plot is a 2-D pass/fail map across two operating parameters (classically supply voltage $\times$ clock frequency), shading where the part works. It’s a *design characterization* tool — it shows the design is stable across process and “can be manufactured with virtually zero yield loss.” You produce shmoos in DV; you do not shmoo every unit on the line.
- Margining is the continuous-parameter cousin: step an operating point (sampling time/voltage, a TX preset) until errors appear and record *how much margin* there was. In DV you margin across corners to characterize; in MT you margin once at nominal and compare to a limit. PCIe lane margining (PCIe chapter) is exactly this.
- Go/no-go is the MT default: run, compare each measured value to its limit, emit PASS/FAIL. Fast, repeatable, operator-runnable.

4.3 The unifying idea: capture the parameter, not just the verdict

This is the single most important design principle for your test code, and the literal reason the JD pairs “capture test parameters” with “analyze results for continuous improvement”:

Capture the parameter, not just the verdict. In DV you sweep and *plot* the captured parameter (the shmoo, the margin-vs-temperature curve). In MT you compare that *same* captured parameter to a limit for a fast pass/fail. Same measurement code, same captured field; the only difference is whether you sweep-and-plot (DV) or compare-to-limit (MT).

Concretely with this toolkit:

- The BERT measures (errors, bits) $\rightarrow$ a BER upper bound. DV: run it across voltage/temperature corners and TX presets and *plot the surface*. MT: run it once to a confidence target (prove $\text{BER} < 1\text{e-}12$ at 95% and stop) and emit pass/fail. *Same engine*.
- Lane margining yields a per-lane timing margin in UI. DV: sweep it across temperature to characterize the eye and *set* the limit. MT: compare the one nominal number to that limit.

The lane-margining number is the bridge: DV uses it to *set* a data-driven per-lane eye limit; MT uses it to *check* that limit per unit — replacing pass/fail-on-link-up with a margin number. That’s the senior-level pitch in one sentence, and it’s why MT must capture parameters: you cannot set a good limit on data you didn’t keep (and the captured stream is what later feeds SPC, C_{pk} , and guard-banding — see Test Economics).

5 PCIe, At the Register Level

The study guide covers PCIe architecture, the LTSSM, generations, and troubleshooting scenarios (§4.1–4.15) well — read it for the conceptual frame. This chapter assumes all of that and goes where the study guide stops: the actual registers you read and write, the write-1-to-clear discipline your BERT tool depends on, lane margining (the modern, scope-free eye measurement), and the BER confidence math. This is the “strong PCIe troubleshooting, especially GPUs” bonus qualification, made concrete.

Companion: Guide C, the PCIe Diagnosis & Root-Cause Playbook. This chapter teaches the registers and the BERT; Guide C (**03-pcie-diagnosis-playbook.md**) is the bench artifact — a decision tree from symptom to root-cause layer (SI vs protocol vs power vs thermal vs firmware) with the exact command sequences, a worked header-log decode, the LBMS/LABS/DLLA latches, the Completion-Timeout/ASPM A/B test, DPC handling, retimer/switch tree-walking, and a symptom→evidence→cause master table. Read this chapter for the *why*; keep Guide C open when a board is on the fixture.

5.1 Config space is just a file you can read

On Linux every PCIe function exposes its config space as a file:

```
/sys/bus/pci/devices/0000:03:00.0/config      # raw config space (read with pread)
/sys/bus/pci/devices/0000:03:00.0/current_link_speed  # "16.0 GT/s"
/sys/bus/pci/devices/0000:03:00.0/current_link_width  # "16"
/sys/bus/pci/devices/0000:03:00.0/max_link_speed      # capability ceiling
/sys/bus/pci/devices/0000:03:00.0/max_link_width
```

You can read every register without `setpci` by `pread`-ing `config` at the right offset — which is exactly what the toolkit’s C engine does (no fragile text parsing of `lspci` output). The first 256 bytes are legacy config space; 4096 bytes total with extended config space (where AER lives). To get there you need root and you read the binary file directly.

The layout you navigate:

0x000	+-----+ Type 0/1 header	Vendor/Device ID, Command, Status, BARs...
0x040	+-----+ Capability list	linked list via "next pointer" bytes:
	- PCIe Capability (0x10)	-> Link Cap/Ctrl/Status live here
	- MSI / MSI-X	
	- Power Management	
0x100	+-----+ Extended Capability list	<- Extended config space (PCIe-only)
	- AER (ID 0x0001)	another linked list:
	- Secondary PCIe (0x0019)	-> the error registers
	- Lane Margining (0x0027)	-> Gen3+ equalization control
	- DPC, ACS, SR-I/OV, ...	-> receiver eye margin per lane
0xFFFF	+-----+	

How to find a capability: capabilities are a linked list. Legacy caps start at the byte pointed to by config offset 0x34; each cap has `[cap_id][next_ptr]`. Extended caps start at 0x100; each has a 32-bit header `[cap_id:16][version:4][next_ptr:12]`. Walk the next pointers until you find the ID you want (AER = 0x0001). The toolkit does this walk for you; doing it by hand once cements it.

5.2 AER: the registers that catch link errors

The AER (Advanced Error Reporting) extended capability is where correctable and uncorrectable PCIe errors are latched. Relative to the AER capability base:

Offset	Register	Notes
+0x00	Capability Header	Cap ID 0x0001
+0x04	Uncorrectable Error Status	W1C. Non-zero after test = FAIL
+0x08	Uncorrectable Error Mask	1 = this error is masked (not reported)
+0x0C	Uncorrectable Error Severity	1 = fatal, 0 = non-fatal
+0x10	Correctable Error Status	W1C. Recovered errors; high count = SI concern
+0x14	Correctable Error Mask	1 = masked
+0x18	Advanced Error Cap & Control	First-error-pointer, ECRC enable bits
+0x18	Advanced Error Cap & Control	First Error Pointer [4:0] (which uncorrectable bit the header log belongs to), ECRC enable bits
+0x1C	Header Log (16 bytes)	First 4 DWORDs of the TLP that caused the first uncorrectable error
+0x2C	Root Error Command	Root ports / RCEC only
+0x30	Root Error Status	Root ports only
+0x34	Error Source ID	Root ports only — requester ID of the COR/UNCOR source

Root-port-only caveat. The Root Error Command/Status and Error Source ID at +0x2C/+0x30/+0x34 exist only on Root Ports and Root Complex Event Collectors, not on endpoints or switch downstream ports. When you want “which requester caused this,” read the Error Source ID on the root port above the device, not on the device — the root port is where the kernel AER driver assembles the OS-level error story. (Playbook Guide C §4 has the worked detail.)

Correctable Error Status bits (these are your signal-integrity early-warning system):

Bit	Error	What it usually means
0	Receiver Error	Physical-layer symbol/8b10b/128b130b error — raw SI problem
6	Bad TLP	A TLP arrived with bad LCRC/sequence — got NAK’d and replayed
7	Bad DLLP	A DLLP (ACK/NAK/flow-control) was corrupted
8	REPLAY_NUM Rollover	The replay counter wrapped — lots of retries happening
12	Replay Timer Timeout	No ACK in time → replay. Classic marginal-link symptom
13	Advisory Non-Fatal	An uncorrectable error was demoted to advisory
14	Corrected Internal Error	Device-internal corrected error
15	Header Log Overflow	More errors than the log could hold

Uncorrectable Error Status bits (any of these set after a clean run = the unit fails):

Bit	Error	Typical root cause
4	Data Link Protocol	Sequence/ACK protocol violation

Bit	Error	Typical root cause
5	Surprise Down	Link dropped unexpectedly (device/power lost)
12	Poisoned TLP Received	Upstream sent data marked bad (EP)
13	Flow Control Protocol	Credit/flow-control violation
14	Completion Timeout	A read never got its completion — hang/upstream issue
15	Completer Abort	Target refused the request
16	Unexpected Completion	A completion with no matching request
17	Receiver Overflow	Receiver buffer overran — flow-control/credit bug
18	Malformed TLP	Structurally invalid packet
19	ECRC Error	End-to-end CRC mismatch (if ECRC enabled)
20	Unsupported Request	Target doesn't support that request
21	ACS Violation	Access Control Services blocked a peer-to-peer TLP
22	Uncorrectable Internal	Device-internal uncorrectable error

The reason this table matters: the bit tells you the layer. Receiver Error / Bad TLP / Replay Timer cluster → physical-layer SI (reseat, temperature, margining, equalization). Completion Timeout / Malformed / Unexpected Completion → transaction-layer/protocol (upstream device, switch, firmware). Your tool should never just say “errors=5” — it should say *which* bits, because that’s the first fork in the debug tree.

5.3 The write-1-to-clear discipline (your tool’s core primitive)

AER status bits are write-1-to-clear (W1C): writing a 1 to a bit clears it; writing 0 does nothing. This has two consequences that drive the tool design:

1. You must clear (arm) the registers before a measurement run. If you don’t, you’re counting errors from boot, power-on glitches, enumeration, and previous tests — not from *your* stress window. Arm → stress → read is the only valid sequence.
2. Clear only the bits that are set, and verify. The elegant W1C trick: read the status register, then write the value you just read back to it. Because only the set bits are 1 in that value, the write clears exactly those bits and touches nothing else — which is precisely the “only if the bits are set” requirement. Then re-read and confirm it’s 0. Fast (one read, one write, one verify), and it never blindly hammers reserved bits.

```
// Toolkit primitive (conceptual). Real version pread/pwrite on /sys/.../config.
uint32_t sts = aer_read(fd, aer_base + AER_COR_STATUS);
if (sts != 0) { // only act if something is set
    aer_write(fd, aer_base + AER_COR_STATUS, sts); // W1C: write back the set bits
    uint32_t after = aer_read(fd, aer_base + AER_COR_STATUS);
    if (after & sts) { /* a bit refused to clear -> sticky/persistent error, report it */ }
}
```

Some flows clear with `setpci -s <bdf> ECAP_AER+0x04.L=0xFFFFFFFF` (write-all-ones, clears everything). That works because writing 1 to an already-clear or reserved bit is harmless, but write-back-the-read-value is the precise, auditable form — it only ever clears what was set, and a bit that won’t clear is itself a finding (a stuck/ persistent error, e.g., a hardware fault latching every cycle). The toolkit uses the read-modify form and treats “won’t clear” as a distinct failure mode.

A subtlety worth knowing: clearing the *AER* status bits does not clear the legacy Device Status error summary bits (in the PCIe Capability) the same way, and the OS's AER driver may also be consuming/clearing errors out from under you via the `dmesg` path. For deterministic manufacturing measurement you typically (a) make sure the kernel `pcie_aer` reporting won't race you, and (b) own the arm/clear yourself right before the stress window. The toolkit documents both the "let the OS report" mode and the "I own the registers" mode.

5.4 Link speed, width, and reading the LTSSM

The negotiated link state lives in the PCIe Capability (not the AER cap):

Register	Offset (rel. to PCIe Cap)	Key fields
Link Capabilities	+0x0C	Max speed [3:0], max width [9:4], DLL Link Active Reporting Capable (bit 20)
Link Control	+0x10	ASPM ctrl, Retrain Link (bit 5), link disable
Link Status	+0x12	Current speed [3:0], current width [9:4], Link Training (bit 11), DLLLA (bit 13), LBMS (bit 14), LABS (bit 15)
Device Control 2	+0x28	Completion Timeout Value [3:0] (the CTO timer range)
Link Control 2	+0x30	Target link speed (used to force a gen for retrain)
Link Status 2	+0x32	Flit Mode Status (bit 10) — set when the link is in Gen6 FLIT mode

Speed encoding in those 4 bits: 1=2.5 (Gen1), 2=5 (Gen2), 3=8 (Gen3), 4=16 (Gen4), 5=32 (Gen5), 6=64 GT/s (Gen6). The easiest read of the current speed/width is `sysfs (current_link_speed/current_link_width)`, which reflects `LnkSta +0x12 [3:0] / [9:4]`; knowing the register lets you read it even when `sysfs` is stale or on a device behind a switch.

The latched status bits are the ones a snapshot misses. A 5 ms poll on "is it Gen4 x16 now" misses a link that bounced to Recovery and back between polls. `LnkSta` gives you latches that survive between reads: DLLLA (bit 13) drops on a link-down (valid only if `LnkCap` bit 20, DLL-Link-Active-Reporting-Capable, is set); LBMS (bit 14, W1C) sets when the link changed speed/width via a *managed* retrain; and LABS (bit 15, W1C) sets when the hardware changed speed/width autonomously — i.e. it could train to the high rate but couldn't *hold* it. **LABS** latching after a soak is the canonical "trains fine, marginal under load" catch that a one-shot speed check passes wrongly. Arm them by writing 1s (`setpci -s <bdf> CAP_EXP+0x12.W=0xc000` clears LBMS|LABS), soak, then re-read.

Gen6 changes the error model (`LnkSta2` Flit-Mode). When `LnkSta2 +0x32` bit 10 (Flit Mode Status) is set, the link runs in FLIT mode: DLLPs are gone (ACK/NAK and flow-control move *inside* the 256-byte FLIT), and errors are caught by FEC (correctable symbols) + a strong CRC + replay instead of LCRC-retry. An AER-correctable BERT *under-measures* a Gen6 link — a true Gen6 BERT must read FEC correctable/uncorrectable counters. Check that bit before trusting LCRC-retry-based error accounting on Gen6 silicon.

The full LTSSM state (Detect/Polling/Config/L0/Recovery/...) is generally not exposed in standard config space — it's in vendor-specific registers (PLX/Broadcom, Intel, etc.) or via a PHY/debug interface. What you *can* observe portably:

- Link-training bit (Link Status bit 11) flicking to 1 = the link entered Recovery and is retraining. Poll it during a stress run; frequent retrains = marginal SI even if the speed/width look fine afterward.

- dmesg: the kernel logs link-down/up, speed changes, and AER events.
- Recovery entry counters on devices/switches that expose them (some Broadcom switches and NVIDIA GPUs do, via vendor registers / `nvidia-smi`).

This is why the toolkit’s PCIe monitor doesn’t just snapshot speed/width once — it *watches the training bit and AER counters over the stress window* and reports retrain events, not just the final state. A link that ends at Gen4 x16 but retrained 40 times during the soak is a failing unit; a one-snapshot test would pass it.

5.5 Equalization, presets, and why links fall back

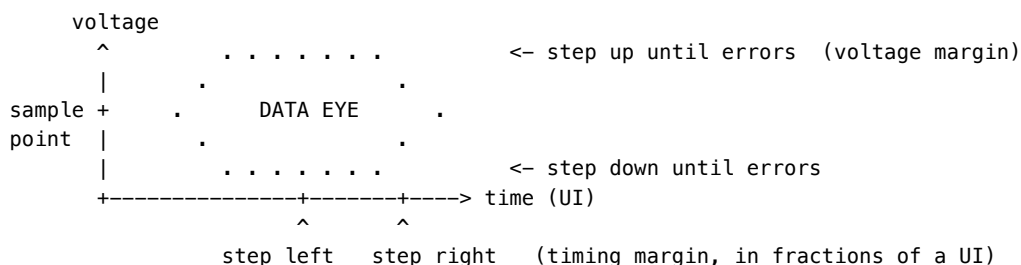
At Gen3 (8 GT/s) and above the channel — PCB traces, vias, connectors, cables — attenuates the high-frequency content of the signal so badly that the raw eye is closed. Equalization reopens it. The study guide (§4.8–4.9) covers the 4-phase handshake and your X-ES pre-emphasis work; the operational points you need on the job:

- TX equalization shapes each bit with three coefficients — pre-cursor (C_{-1}), cursor (C_0), post-cursor (C_{+1}) — under a fixed-swing constraint $|C_{-1}| + |C_0| + |C_{+1}| = \text{const}$. More emphasis on the cursors = stronger transitions but a weaker main bit. Too much → overshoot/ringing; too little → closed eye.
- The spec defines 11 TX presets (P0–P10) that bundle coefficient ratios for different channel losses. During training the hardware negotiates a preset per lane; in manufacturing you sometimes override or characterize the best preset for a specific board revision (your X-ES sweep).
- RX equalization: CTLE (continuous-time linear EQ, frequency-dependent gain) + DFE (decision-feedback EQ, cancels ISI using past bit decisions). The receiver adapts these during training; you usually don’t set them directly, but they’re why a “bad” TX preset can still sometimes train — the RX compensates until it can’t.
- Retimers vs redrivers: long channels (a board-to-board cable, a riser) often insert a retimer (a protocol-aware repeater that re-clocks and re-equalizes — it shows up as a PCIe device and has its own link/error state) or a redriver (an analog booster, invisible to software). On Zoox’s custom boards with cables, expect retimers. They add link segments that each train independently and each have their own failure modes — your topology map needs to include them.

Why a link falls back to a lower speed/width (the single most common PCIe finding): equalization failed to reach a usable eye at the higher gen, so the LTSSM negotiated down. Causes: trace/length, via stubs, impedance discontinuities, connector seating, temperature (a board that equalizes fine at 25 °C can fail at 55–85 °C), and power integrity. The triage table at the end of this chapter maps symptoms to causes.

5.6 Lane Margining at the Receiver (the scope-free eye)

This is the modern technique that turns your X-ES “sweep presets and run a BERT” into a spec-standard, push-button measurement — and it’s a genuinely strong thing to bring to Zoox. Lane Margining at the Receiver is a mandatory PCIe Gen4+ feature (Extended Capability ID `0x0027`). It lets software command the receiver to shift its sampling point — in time (left/right of the data eye) and, on capable receivers, in voltage (up/down) — per lane, and report when errors start. Stepping the offset until errors appear measures the eye margin directly, on-die, with no oscilloscope.



The access path is per the spec’s Margining Lane Control/Status registers in that capability (you issue “set timing offset = N steps”, “go”, then read the error count / “too many errors” status, per lane). The real Linux tool that does this today is **pci_lmr** — part of `pciutils` (≥ 3.13 , May 2024), no vendor SDK required. There is no generic

kernel sysfs “margin this lane” interface; `pcilmr` drives the capability registers from user space (and hardcodes vendor quirks like Ice Lake that a hand-rolled sequence won’t).

```
sudo pcilmr --scan                # list links that can be margined (negotiated >=16 GT/s)
sudo pcilmr --margin -TV 0000:03:00.0 # all lanes, timing (T) + voltage (V)
sudo pcilmr --margin -TV -r 1,2,3,6 0000:03:00.0 # near RX, retimer RXs (2-5), far RX (6)
sudo pcilmr -o ./csv --full        # margin every ready link, CSV out for limit-setting
```

It needs root, the link in D0, and ASPM/HW-autonomous features disabled during the test (`pcilmr` does the latter and warns). The per-lane result converts to UI/mV as $\text{margin_UI} = (\text{passing_steps}/\text{NumTimingSteps}) \times (\text{MaxTimingOffset}/100)$; spec eye targets it grades against are ~30% UI minimum (38% recommended) and ~15 mV at 16 GT/s. The toolkit ships a mock margining backend so you can demo the sweep on your laptop; the robust real-hardware path is to shell out to `pcilmr` and parse its CSV (richer alternatives: OCP `pci_lmt`, Google `pcie_lmt`, Oxide `lmar`). Either way the output is the per-lane timing/voltage margin number — the manufacturing metric you actually want (“every lane has $\geq X$ UI of timing margin”) instead of the binary “it trained.” (Guide C, the PCIe Diagnosis Playbook, has the full margining + retimer-localization workflow.)

Why this lands at Zoox. Their bonus qual is “strong PCIe troubleshooting.” Walking in able to say “I’d add receiver lane margining to module test so we get a per-lane eye margin number and can set a data-driven limit, instead of pass/fail on link-up” is a senior-level coverage improvement, not a script. It’s also the natural, spec-blessed successor to the pre-emphasis sweep you already did at X-ES.

5.7 Decoding the AER Header Log (turning a bit into a sentence)

When an uncorrectable error latches, AER captures the first 4 DWORDs of the offending TLP in the Header Log (+0x1C), and the First Error Pointer (`ERR_CAP[4:0]`, +0x18) says which uncorrectable bit that log belongs to. Decoding it turns “Completion Timeout” into “a 4-byte memory read of address 0x05010000 by requester 00:04.0 timed out” — which tells you to suspect the *completer* of that address, not the requester’s link. DW0 carries Fmt/Type (read vs write, mem vs config vs completion, 3- vs 4-DWORD header); DW1 carries Requester ID/Tag/byte-enables; DW2/DW3 carry the address (or, for a completion, Completer ID + status + byte-count). `lspci -vvv` and the kernel `dmesg` AER line both print the four DWORDs raw. The toolkit captures the header log + First Error Pointer for every uncorrectable so a failure arrives with its requester/type/address already decoded — the single highest-value diagnostic addition over “errors=N”. (Guide C §4 is a full worked decode.)

5.8 DPC — when the device vanishes on purpose

Downstream Port Containment (DPC, extended cap ID 0x001D) is a root/switch downstream-port mechanism that, on a Fatal/Non-Fatal error, automatically disables the link to contain the error. The visible symptom is the device disappears and `dmesg` shows DPC containment, then the kernel attempts recovery and re-enumerates. For manufacturing test it’s a double-edged sword: it cleanly *captures and isolates* a fatal event (the `DPC_STATUS` trigger reason tells you *why* — uncorrectable / `ERR_NONFATAL` / `ERR_FATAL` / `RP-PIO` / `SW-trigger`, and `RP_PIO_*` logs the offending root-port read) but it also yanks the device out from under your test. Know whether DPC is enabled on your root ports (`lspci -vvv` → `DPC: ... Enabled`) and decide per station whether to leave it on (captures + isolates) or off (keep stressing past the first fatal). This is why “the device vanished” must be read correctly: DPC containing a real fatal error looks different from a power glitch or a hot-unplug.

5.9 Self-testing the AER pipeline (`aer-inject`)

You can validate your whole AER decode/clear/count path with no bad hardware using `aer-inject` (needs a kernel with `CONFIG_PCIEAER_INJECT`). Inject a known correctable (`BAD_TLP`) and a known uncorrectable with a header log, then assert your tool reports exactly that bit, the matching header log, the right First Error Pointer, and a clean W1C clear. It’s the AER analog of a golden-unit correlation, and a strong bring-up/interview artifact: “my tool’s AER path is self-tested against a known input.” (Guide C §13 has the recipe.)

5.10 The BERT: measuring bit-error rate to a confidence level

This is your tool's core. The goal of a manufacturing BERT isn't to measure the *exact* BER — it's to prove the BER is below a target (1e-12) with a stated confidence in the shortest time. The statistics:

Model errors as Poisson with mean $\lambda = n \cdot p$, where n = bits transferred and p = the target BER. The confidence that the true BER is below p , given E errors observed:

$$CL = 1 - \sum_{k=0}^E e^{-np} \frac{(np)^k}{k!} = 1 - \text{PoissonCDF}(E; np)$$

Two results fall out of this and run the whole test:

- Zero-error case (the common one): set $E=0$ and solve for the bits needed —

$$n = \frac{-\ln(1 - CL)}{p}.$$

For 95% confidence at $p = 1e-12$: $n \approx 3.0 \times 10^{12}$ bits (the “3 / BER” rule of thumb, since $-\ln(0.05) \approx 2.996$). That's the target your tool transfers cleanly.

- Reporting the bound (for any error count): the exact upper bound on BER you can claim at confidence CL after n bits with E errors is

$$\text{BER}_{\text{upper}}(CL) = \frac{\chi_{\text{inv}}^2(CL, 2E + 2)}{2n},$$

which reduces to the zero-error formula when $E=0$. This is why the math is in Python (incomplete-gamma / chi-squared inverse, with arbitrary precision as a fallback) while the C engine just counts errors and bits — exactly the split you used at X-ES.

Turning bits into time. The engine accounts n either by *measuring* throughput of a real workload, or *theoretically* from the link: $n \approx \text{link_speed} \times \text{link_width} \times \text{efficiency} \times \text{time}$. Sanity points: Gen4 x16 ≈ 31.5 GB/s $\approx 2.5 \times 10^{11}$ bit/s, so 3×10^{12} bits $\approx \sim 12$ s of clean traffic; an NVMe Gen3 x4 link ≈ 3.5 GB/s, so $\approx \sim 110$ s. Those are realistic module-test durations, which is the point of choosing a confidence target rather than running forever.

The full BERT loop your tool runs: (1) arm — clear AER status with the W1C read-back primitive and confirm zero; (2) stress — drive traffic (or account theoretical bits) for an interval; (3) read — sample AER correctable/uncorrectable counts and the link-training bit; (4) accumulate n and E ; (5) decide — feed n , E , p , CL to the Python layer; stop when $CL \geq \text{target}$ (PASS) or any uncorrectable error appears or the time budget expires (FAIL/inconclusive). The diagnostics (which AER bits, retrain count, margin) ride along so a failure comes with its root-cause evidence already attached.

5.11 Bringing up a *custom* PCIe device

The JD says “custom PCIe devices.” For an off-the-shelf GPU you have NVIDIA's tools; for Zoox's own card you have a schematic and a problem. The bring-up order you'll use:

1. Does it enumerate? `lspci -nn` — right Vendor/Device ID at the expected BDF? If not: power rails, REFCLK present, PERST# released, the LTSSM stuck in Detect/Polling (no partner / SI), or a fundamental layout error. Check `dmesg` for training failures.
2. Right class/BARs? `lspci -vvv -s <bdf>` — are BARs assigned (not “ignoring BAR / can't assign”) ? Unassigned BARs = the device is dead to software even though it's visible.
3. Right link? Current vs max speed/width. A brand-new board that trains x16→x8 or Gen4→Gen3 on the bench is your first SI finding.
4. Does it respond? Read a known register over MMIO; for a custom FPGA card, the design team gives you a scratch/ID register to prove the datapath.

5. Driver / interrupts? Does the driver bind (`lspci -k`)? Are MSI/MSI-X vectors allocated and is `/proc/interrupts` counting during activity? A device that looks fine in `lspci` but never interrupts is a dead-on-arrival from the OS's view.
6. Errors under stress? Now your BERT/AER monitor, hot and cold.
7. Margin? Lane margining for the per-lane eye.

For custom cards you'll often work *from the schematic*: which CPU root port feeds this slot, is the slot bifurcated (one x16 split into 2x8 or 4x4 — a top cause of “device not detected” when BIOS bifurcation doesn't match the layout), where are the retimers, which rails feed the PHY. Bring-up is where you and EE are closest; your job is to produce evidence sharp enough that a layout or stuffing fix is obvious.

5.12 PCIe failure-signature triage

Symptom	Most likely cause	First moves
Trained below max speed (Gen4→Gen3)	Equalization/SI margin, BIOS gen cap, thermal	Compare LnkCap both ends; <code>dmesg</code> gen change; retest hot/cold; lane margining
Trained below max width (x16→x8)	Connector seating, bent pin on high lanes, bifurcation, lane reversal	Reseat; per-lane margining to find the dead lane; check BIOS bifurcation
High correctable count (Bad TLP / Replay Timer)	Physical-layer SI, marginal lane, temperature	Decode which bit; thermal soak; margining; reseat; check rails
Completion Timeout uncorrectable	ASPM/L1-exit latency <i>or</i> hung/mis-addressed completer	Run the <code>pcie_aspm=off</code> A/B test first (vanishes → L1 latency; persists → completer); decode header log; check DEVCTL2 CTO value
Other uncorrectable (Malformed / UnexpCmpl / FCP)	Upstream/switch/firmware, protocol/IP bug	Check upstream device + switch; <code>dmesg</code> ; header-log decode; bisect FW/driver
Frequent retrains (training bit toggles), LABS latched	Marginal SI even if final state is good	Watch latches over soak; margining; correlate with temp/power
Device not detected at all	Power/REFCLK/PERST#, bifurcation, dead PHY	Rails, <code>dmesg</code> LTSSM stuck state, BIOS bifurcation, reseat
Detected but driver won't bind	Wrong/blacklisted module, BAR alloc fail, device in D3	<code>lspci -k</code> , <code>dmesg <bdf></code> , check Status reg, power state
Errors only under load/heat	Thermal SI / power droop	Stress + AER, scope the rail, thermal chamber

5.13 Gen5/Gen6, switches, and what's coming

- Gen5 (32 GT/s) is in current server silicon and almost certainly in Zoox's roadmap; margins are tighter, equalization and lane margining matter more, and retimers are common. Gen6 (64 GT/s) moves to PAM4 signaling (4 levels instead of 2) with FEC — a different error model (FEC corrected vs uncorrected symbols), which changes what “BER” even means. You don't need Gen6 day one, but know it's coming and that PAM4+FEC reshapes the diagnostics.
- PCIe switches (Broadcom/PLX, Microchip) fan one root port out to many endpoints — exactly the “custom switch/fan-out board” case. Each downstream port is its own link with its own LTSSM, AER, and equalization. Your topology map and your error monitor must walk every port (`lspci -tv`), not just the endpoints behind it — a fallback or error cluster can be on *any* segment, and AER's Error Source ID (on the root port) attributes it to the true requester. (Your X-ES PLX experience is directly relevant here.)

- Retimers (protocol-aware repeaters, up to 2 per link) reset the jitter/loss budget and create an independent link segment on each side — and crucially they appear in lane margining as additional Receiver Numbers (pcilmr -r 2..5). That lets you margin the retimer's RX and localize a marginal eye to “before vs after the retimer” (board trace vs cable) — a huge lever on Zoox's cabled board-to-board links. A redriver, by contrast, is an analog booster: invisible to software, no link state, can't be margined — if a link has one and a marginal eye, you're back to the scope.

6 NVMe Storage, For Test Engineers

NVMe is a PCIe endpoint, so everything in the PCIe chapter applies first — an NVMe drive that throws Bad-TLP correctable errors or trains x4→x2 is a PCIe problem wearing a storage costume. The study guide §5 covers the architecture and `nvme-cli` basics. Here's the depth you need to *qualify* a drive on the line.

6.1 The model in one paragraph

The host and controller communicate through queues in host memory: Submission Queues (host writes commands, rings a doorbell register) and Completion Queues (controller writes results, raises an MSI-X interrupt). There's one Admin queue (identify, get-log, firmware, format, self-test) and many I/O queues (read/write/flush) — typically one per CPU core, which is how NVMe gets its parallelism. A drive exposes one or more namespaces (`/dev/nvme0n1`) under a controller (`/dev/nvme0`). When you test, you test both: the controller (health, firmware, self-test) and the namespace (data integrity, performance).

6.2 SMART / Health — your pass/fail gate

`nvme smart-log /dev/nvme0 -o json` is the single most important command. The manufacturing limits on a new drive:

Field	New-drive expectation	Why
<code>critical_warning</code>	0	Any bit set = failing (spare low, temp, read-only, volatile-memory-backup failed)
<code>media_errors</code>	0	Uncorrectable media errors — a brand-new drive should have none
<code>num_err_log_entries</code>	0 (or known-benign)	Total error count
<code>percentage_used</code>	0%	Wear indicator; non-zero on “new” = used/returned stock
<code>available_spare</code>	100%	Spare blocks remaining
<code>available_spare_threshold</code>	below current spare	If spare < threshold, <code>critical_warning</code> trips
<code>temperature</code>	within spec (often 0–70 °C)	Out of range during test = thermal/airflow problem
<code>data_units_written/read</code>	small/reasonable	Large values on “new” = the drive has history
<code>power_on_hours</code>	single-digit hours	Re-stock detector — a “new” drive with hundreds of hours is re-used/RMA stock
<code>power_cycles</code>	low	Same re-stock signal as power-on-hours
<code>unsafe_shutdowns</code>	typically 0	Context for field returns
<code>thm_temp1/2_trans_count</code>	0	Nonzero = the drive is already throttling at bring-up → thermal/airflow/mount problem
<code>warning_temp_time / critical_comp_time</code>	0	Minutes spent over WCTEMP/CCTEMP — nonzero on a new drive is a thermal finding

A subtlety that catches people: `percentage_used`, `data_units_written`, **`power_on_hours`**, and **`power_cycles`** non-zero on a “new” drive is how you catch re-labeled or returned stock entering your line — a supply-chain/quality finding, not a drive fault. Your test should log these even when they pass, so the fleet data (and

genealogy) can flag a bad lot. Equally, any nonzero `thm_temp*_trans_count` or `warning_temp_time` means the drive throttled *during your own test* — a cooling/airflow/mounting problem, not a drive defect per se.

Tool note: the current `nvme.py` gates on `percentage_used < 2` and `available_spare >= 100` (good) but does not yet check `power_on_hours/power_cycles` (re-stock), the thermal-throttle transition counters, or `unsafe_shutdowns` — those are the high-value adds for catching used stock and already-throttling drives.

6.3 The commands that matter beyond SMART

```
nvme list -o json                # enumerate: model, serial, FW, namespace
nvme id-ctrl /dev/nvme0 -o json  # controller identify (model, FW, features, limits)
nvme id-ns /dev/nvme0n1 -o json  # namespace identify (size, LBA format, metadata)
nvme error-log /dev/nvme0 -o json # recent command errors (should be empty/benign)
nvme fw-log /dev/nvme0          # firmware slots + active slot
nvme get-feature /dev/nvme0 -f 0x04 -H # temperature thresholds (TMT1/TMT2)
nvme device-self-test /dev/nvme0 -s 1 # start SHORT self-test (controller's own POST)
nvme device-self-test /dev/nvme0 -s 2 # EXTENDED self-test (longer, more coverage)
nvme self-test-log /dev/nvme0      # poll result: percent complete, pass/fail
```

The Get Log Page surface (the diagnostic logs beyond SMART) — these are what a credible NVMe qualification reads, not just `smart-log`:

LID	Log	What it gives you
0x01	Error Information	Ring of recent error entries (status code, command ID, LBA, NSID) — more detail than SMART's <code>num_err_log_entries</code> summary
0x02	SMART / Health	The core health page above
0x03	Firmware Slot Info	Active/next slot + per-slot revision strings
0x06	Device Self-Test	Result of the last self-tests (up to 20 entries) + current-operation % complete
0x07/0x08	Telemetry (Host / Controller-Initiated)	Vendor binary blobs for FA/RMA (<code>nvme telemetry-log</code>)
0x0D	Persistent Event Log	Non-volatile, cross-power-cycle history (power cycles, thermal excursions, firmware changes, errors) — the richest field-return artifact

Device Self-Test (DST) is underused and valuable: the controller runs its own internal diagnostic (read/verify across the media, internal checks) and reports pass/fail. It's "free" coverage you didn't have to write — but the gotcha is DST is non-blocking: `nvme device-self-test ... -s 1` only *starts* it and returns immediately. You must then poll log page 0x06 (`nvme self-test-log`) for completion % and the pass/fail result code of the latest entry. A test that starts a DST and never reads 0x06 gives *false assurance* — it proved nothing. Start a short DST early, then read the 0x06 result at the end while your other tests run in parallel.

The non-volatile logs are the RMA story. SMART resets some context; the Persistent Event Log (0x0D) and Error Information Log (0x01) carry the history a fresh SMART page hides — prior thermal excursions, firmware changes, error bursts. Capture both (and `telemetry-log` on a failure) into the test record so failure-analysis and quality have the non-volatile history, not just the moment-of-test snapshot. (And note: a sanitize clears some drives' logs — capture logs *before* any erase.)

6.4 Stress and data integrity with **fio**

SMART tells you the drive’s opinion of itself; fio makes you form your own. The recipes you’ll keep:

```
# Sequential write throughput (does it hit rated BW? does it throttle/heat?)
fio --name=seqwrite --filename=/dev/nvme0n1 --rw=write --bs=128k \
    --iodepth=32 --numjobs=1 --direct=1 --runtime=120 --time_based --group_reporting

# Random read IOPS (the queue-depth/parallelism test)
fio --name=randread --filename=/dev/nvme0n1 --rw=randread --bs=4k \
    --iodepth=256 --numjobs=4 --direct=1 --runtime=120 --time_based --group_reporting

# DATA INTEGRITY: write a known pattern, read it back, verify every byte
fio --name=verify --filename=/dev/nvme0n1 --rw=write --bs=64k --direct=1 \
    --verify=crc32c --verify_fatal=1 --do_verify=1 --size=4G
```

Destructive-test discipline. Writing to /dev/nvme0n1 (or nvme format / secure erase) destroys data. On the line that’s fine — the drive is blank. On any shared or dev machine it’s a disaster. Every destructive test in your toolkit must require an explicit “yes, this device, I mean it” confirmation and refuse to run against a mounted filesystem or the boot drive. Treat this like a loaded tool.

What to watch *during* the soak: throughput/latency holding steady (a cliff = thermal throttling), nvme smart-log composite temperature vs the thermal thresholds (TMT1 triggers light throttle, TMT2 heavier), and — because it’s a PCIe device — AER counters on its link. A drive that hits rated BW for 10 s then halves it has a thermal/airflow problem that only module-level soak (not PCBA) will catch.

6.5 Firmware update flow (you will own this)

“Release updated test programs for new generations of hardware” includes flashing drive firmware on the line:

```
nvme fw-download /dev/nvme0 --fw=image.bin      # stage the image (chunks to the controller)
nvme fw-commit   /dev/nvme0 --slot=1 --action=1 # commit to slot 1, activate on next reset
# then a controller reset (or power cycle) to run the new FW; re-read fw-log to confirm
```

The gotcha is the activation/reset semantics (action codes: download-only vs activate-on-reset vs activate-immediately) and that some drives need a full power cycle, not just a controller reset. Your test must confirm the new version is *running* (fw-log active slot), not just downloaded.

6.6 Form factors and therm/hot-swap

M.2 (consumer/edge), U.2 / U.3 (2.5” enterprise, hot-swappable, used in servers), and EDSFF (E1.S / E1.L / E3) (modern data-center rulers). Zoox’s server-grade assemblies likely use U.2/U.3 or EDSFF with proper thermal paths. Form factor changes how the drive is cooled (and thus how it throttles), whether it’s hot-swappable (U.2/U.3 — your test may need to verify hot-insert), and the connector you’re qualifying. M.2’s poorer cooling makes thermal throttling a more common finding there.

7 GPUs, For Test Engineers

Same opening note: a GPU is a PCIe device first. `nvidia-smi -q -d PCIE` showing the link at Gen3 when it should be Gen4, or a climbing Replay Count, is a PCIe SI problem found through the GPU's own telemetry. The study guide §6 covers `nvidia-smi/DCGM` and the test sequence; here's the depth.

7.1 ECC — the GPU's error story (analogous to AER)

Data-center GPUs run ECC on their memory and internal RAMs. The distinction that matters:

- Corrected errors (single-bit, SBE) — recovered, like PCIe correctable. A few over a long run can be normal; a high or climbing rate is a marginal-memory finding.
- Uncorrected errors (double-bit, DBE) — *not* recovered, like PCIe uncorrectable. Any on a new unit = fail.
- Volatile vs aggregate counts — and this is a real gating gotcha: volatile resets on reboot/driver reload; aggregate is lifetime (stored in the GPU's InfoROM). Gate the manufacturing pass on **volatile** uncorrected == 0, and treat a nonzero aggregate as an *investigate* / *RMA-history* flag, not an automatic fail — otherwise a perfectly good, already-remapped used GPU fails forever on one historical DBE. So you clear/baseline volatile, stress, then read volatile (arm/stress/read, like AER), and separately *log* aggregate for history. (A common tool bug is querying only `...aggregate.total` and gating on it — wrong on both counts.)
- Row remapping (Ampere+) / page retirement (older): on an uncorrectable (or a threshold of correctables) the GPU retires/remaps the affected row so it's not reused. XID 63 = remap succeeded (reset pending); XID 64 = remap failed. A new GPU with pending or failed remaps is an RMA signal even with zero *live* ECC — `log nvidia-smi -q -d ROW_REMAPPER` (or `retired_pages.*` on older parts) and check the remapped-row count, pending, and failure flags.

```
nvidia-smi -q -d ECC           # volatile + aggregate SBE/DBE counts
nvidia-smi -q -d ROW_REMAPPER # remapped rows / pending (Ampere+)
nvidia-smi -e 1               # ensure ECC is ENABLED (must be, for DC test) - needs reset
```

7.2 Thermal-under-load is the test, not a snapshot

A GPU at idle tells you almost nothing. The defects you're catching — bad heatsink mount, wrong/insufficient thermal paste, dead fan, pump-out, a marginal power stage — only appear under sustained compute load. The sequence:

1. Baseline: `nvidia-smi -q` for temp, clocks (P-state), power, ECC counts cleared.
2. Load: `gpu-burn` or `dcmi diag -r 3` (which includes a real compute + memory stress) for a soak interval.
3. Watch throttle reasons: `nvidia-smi -q -d PERFORMANCE` / `nvidia-smi --query-gpu=clocks_throttle_reasons.active --format=csv`. You want to know *why* clocks dropped: thermal slowdown (hit the temp limit — cooling problem), HW slowdown / thermal violation (hit the hard limit — serious), power brake (hit the power limit — could be normal or a power-delivery issue), or SW power cap.
4. Verify it held up: clocks stayed near P0, temp under the throttle threshold, zero DBE, power draw sane, no new XID errors in `dmesg/nvidia-smi -q -d XID`.

XID errors (in `dmesg` as `NVRM: Xid (PCI:...): NN, ...`) are NVIDIA's catch-all hardware/driver error codes — learning to read them is the GPU analog of decoding AER bits. Each code maps to a root-cause bucket (app vs memory vs bus vs NVLink vs GSP), which is what makes XID monitoring the single highest-signal GPU manufacturing check:

XID	Name	Class / meaning
13	Graphics Engine Exception	Usually app (OOB / illegal instr); run under compute-sanitizer. Rarely HW.
31	GPU memory page fault (MMU)	Usually app illegal address; can be driver/HW.

XID	Name	Class / meaning
43 / 45	GPU stopped / preemptive cleanup	SW-induced teardown after an app abort/SIGKILL; GPU stays healthy.
48	Double-Bit ECC (DBE)	Uncorrectable HW memory error → reset/reboot; repeated → RMA.
62	Internal micro-controller halt	Firmware error → GPU reset.
63	Memory remapping event	Row remap (Ampere+) / page retirement succeeded; reset pending.
64	Memory remapping failure	Remap/retirement failed → reset; possible RMA.
74	NVLink error	Link problem between GPUs / NVSwitch; can be HW.
79	GPU has fallen off the bus	GPU inaccessible over PCIe — often a PCIe link / power / thermal HW failure.
92	High single-bit ECC rate	Excessive SBE (degrading memory).
94 / 95	Contained / Uncontained memory error	94 = isolated to one app (restart it); 95 = affects multiple apps → GPU reset.
119 / 120	GSP RPC timeout / GSP error	GPU System Processor fault → reset.

In burn-in, any XID 48/63/64/79/74/92/94/95 is a hard fail with a clear root-cause bucket. Scrape dmesg/syslog (or the ring buffer) for NVRM: Xid during the soak and bucket by code — that, plus row-remap state below, is the GPU manufacturing gate.

7.3 DCGM is the manufacturing-grade tool

nvidia-smi is for humans; DCGM (Data Center GPU Manager) is for test programs:

```
dcgmi discovery -l      # enumerate GPUs + NVLink topology
dcgmi diag -r 1         # quick (seconds) - sanity
dcgmi diag -r 2         # medium - adds some stress
dcgmi diag -r 3         # long - full memory + compute + stress, the real qualification
```

Level	Flag	What it covers (approx. duration)
1	-r 1	Quick readiness: SW/driver, NVML, basic sanity (~seconds)
2	-r 2	Medium: adds PCIe/NVLink checks, memory bandwidth, integration (~2 min)
3	-r 3	Long: full HW diag + stress — Memory (Targeted), SM/Targeted Power & Stress, PCIe (~several min). The standard qualification run.
4	-r 4	Extra-long (DCGM ≥ 2.4): adds memtest (walking-1s + pattern tests) and the Pulse Test (power-spike PSU stress)

`dcgmi diag -r 3` is close to a turnkey GPU module test: memory tests (catch ECC/memory defects), compute stress (thermal + power), PCIe checks, structured pass/fail (use `-j` for JSON). Your job is to wrap it, set the right plugin thresholds, parse its output (`-j`), fold it into your pytest harness, and add what it *doesn't* cover (your PCIe lane margining, your thermal-correlated AER, XID bucketing during the soak, rail measurements with a real DMM). DCGM complements `gpu-burn` (max-thermal-load soak) — `gpu-burn` for the cooling solution, `-r 3/4` for structured per-subsystem pass/fail — and exposes the NVLink CRC/replay/recovery counters (`dcgmi nvlink --link-status; DCGM_FI_DEV_NVLINK_*`) that XID 74 summarizes.

7.4 Multi-GPU and NVLink (awareness)

Server compute boards run multiple GPUs, sometimes linked by NVLink (a separate high-bandwidth GPU-to-GPU SerDes, not PCIe). `nvidia-smi topo -m` prints the topology (which GPUs share a PCIe switch, which are NVLink-connected); `nvidia-smi nvlink -s` shows per-link status and error counters. You don't need NVLink mastery day one, but know it exists, that it has its own link-up/error state to verify, and that “GPU-to-GPU bandwidth” is a real test (`nvbandwidth` / DCGM) separate from each GPU's PCIe link.

8 Memory (DRAM), For Test Engineers

The JD lists “PC components such as storage and memory.” Memory test has the same shape as everything else here: clear counters → stress (hot) → read counters → decode to a physical part. The counter system for memory is EDAC, and it’s the DRAM analog of AER and GPU-ECC.

8.1 What you’re testing

Server boards use ECC DRAM — RDIMM/LRDIMM with an extra DRAM device per rank so the 72-bit bus carries 64 data + 8 ECC bits, enough for SECCDED (Single-Error-Correct, Double-Error-Detect). High-end memory controllers add Chipkill / SDDC (correct an entire failed DRAM device). DDR5 additionally has on-die ECC — internal single-bit correction inside each chip — but that’s *invisible to the host and does not replace system ECC*; you still rely on the controller’s ECC for the bits on the bus. Know the difference so you don’t mistake “DDR5 has ECC” (on-die, internal) for “system ECC is on” (controller, what you actually monitor).

The error classes mirror everything else:

- CE (Corrected Error) — a single-bit flip the ECC fixed. Like PCIe correctable: a few over a long soak can be benign; a high or climbing rate, or many on one DIMM, is a finding.
- UE (Uncorrectable Error) — ECC detected but couldn’t fix. Like PCIe uncorrectable: any UE on a new unit = fail, and it can crash the box.

8.2 EDAC: reading memory errors in Linux

```
# Raw counters (the memory controller's error registers, surfaced in sysfs)
cat /sys/devices/system/edac/mc/mc0/ce_count      # corrected error count
cat /sys/devices/system/edac/mc/mc0/ue_count      # uncorrectable error count
cat /sys/devices/system/edac/mc/mc0/dimm*/dimm_ce_count  # per-DIMM corrected
cat /sys/devices/system/edac/mc/mc0/dimm*/dimm_label    # which physical DIMM

# rasdaemon - decodes and logs errors and maps them to the DIMM silkscreen label
ras-mc-ctl --summary      # totals of CE/UE seen
ras-mc-ctl --errors       # per-error detail: which DIMM, rank, when
```

The killer feature is **rasdaemon** mapping an error to the DIMM label (e.g., DIMM_A1). When your soak reports corrected errors, you don’t say “the board has memory errors” — you say “DIMM_A1 is throwing 40 CE/hour, swap that stick,” which is an actionable RMA, not a vague fail. Wire `ras-mc-ctl` into your test and log the label every time.

The path from a kernel CE/UE to a silkscreen label: the controller’s `per-dimm*/csrow*` sysfs nodes carry a `dimm_label`, and **rasdaemon** consumes the kernel tracepoints and logs to a SQLite DB. For the label to read DIMM_A1 instead of an abstract `csrow3/channel1`, you populate a board-specific **labels.db** keyed by DMI (the board’s SMBIOS identity) — do this once per board revision and every station inherits the mapping:

```
ras-mc-ctl --error-count    # CE/UE per DIMM (from sysfs)
ras-mc-ctl --summary        # totals of logged errors
ras-mc-ctl --errors         # per-error detail with DIMM labels
ras-mc-ctl --layout         # memory topology
```

The DDR5 masking gotcha. A DDR5 DIMM splits into two independent sub-channels and carries on-die ECC (ODECC) that corrects single-bit errors *inside the die before data leaves the chip*. ODECC can therefore hide a marginal cell from system-level EDAC counters — a DDR5 system can look clean while the die is silently correcting. So system ECC + EDAC remain necessary to catch what escapes on-die correction; don’t read “DDR5 has ECC” as “I can trust a clean EDAC count.” High-end controllers add Chipkill / SDDC (correct an entire failed x4/x8 device via Reed-Solomon symbols) on top — a different, stronger scheme than the 72-bit SECCDED you get on a basic ECC channel.

8.3 Stressing memory

- **stressapptest** (Google's stressful application test) — runs *under Linux*, hammers memory bandwidth and patterns, detects errors by miscompare (writes known data, reads it back), and exercises some I/O too. The manufacturing favorite because it runs inside your normal test environment, hot, for a fixed soak: `stressapptest -M <MB> -s <seconds> -W` (use most of RAM, with memory-copy threads).
- **memtest86+** — boots *instead of* the OS, walks the full address space with thorough patterns. More coverage of pattern-sensitive defects, but it's offline (can't run inside your pytest harness) and slow — usually reserved for a debug bring-up step, not volume line test.
- In-band monitor: whatever stress you run, clear EDAC first and read CE/UE after. The stress *provokes*; EDAC *counts*. Run it hot — DRAM marginality is strongly temperature- and voltage-dependent, so room-temp pass / hot fail is the classic escape.

Pass/fail: zero UE, CE below a data-driven limit, and no error concentrated on a single DIMM/rank above threshold. Like everything else, set the limit from fleet data, not a guess.

9 High-Speed Links: One Mental Model for PCIe, GMSL, and Ethernet

Here's a senior-level insight that ties the JD's interfaces together and will make you faster on the floor: PCIe, GMSL, automotive Ethernet, NVLink, USB, and SATA are all the same kind of thing — serial differential SerDes links. Learn the model once and every new link is a variation:

```
TX --[ encode/scramble ]--[ pre/de-emphasis (TX EQ) ]--+
                                                    | differential pair, AC-coupled
the CHANNEL: PCB trace, vias, connector, cable ~~~~~ (attenuates highs, adds jitter/ISI)
                                                    |
RX --[ CTLE + DFE (RX EQ) ]--[ CDR clock recovery ]--[ decode ]--> bits
+ LINK TRAINING brings the two ends up together; + ERROR COUNTERS catch what slips
```

Every one of these links: recovers a clock from the data (CDR), encodes/scrambles for DC balance and transitions, equalizes (TX emphasis + RX CTLE/DFE) to fight channel loss, trains to bring both ends up, and counts errors. The failure physics are identical — channel loss, jitter, inter-symbol interference, reflections from impedance discontinuities, temperature, and connector/cable quality. So your debugging instincts transfer: *check it's trained at the right rate/width, stress it, watch the error counters, suspect the channel and temperature first*. What changes per link is the vocabulary and the tooling, below.

9.1 GMSL — cameras over coax

Study guide §6.4 has the basics. The operational picture:

```
Sensor (AR0820) --MIPI CSI-2--> Serializer (MAX96717) --coax (+power, P0C)-->
Deserializer (MAX96712, often quad/hex) --CSI-2--> SoC capture/ISP
<==== reverse channel: I2C-over-GMSL configures the camera ====
```

The real SerDes parts (Analog Devices / Maxim — the actual AV silicon):

Part	Role	Capability
MAX9295A/D	Serializer (on camera)	GMSL2/1, single/dual MIPI CSI-2 in; 3/6 Gb/s fwd, 187.5 Mb/s rev
MAX96717 / 96717F	Serializer	GMSL2, CSI-2 in; F = functional-safety variant
MAX96793	Serializer	GMSL3/2, CSI-2 in; 3/6/12 Gb/s fwd (CFG-pin selectable)
MAX9296A	Deserializer	GMSL2/1 → CSI-2; pairs with MAX9295/96717
MAX96712	Quad deserializer	4× GMSL2/1 in (4 cameras) → CSI-2 out; coax or STP
MAX96792A	Dual deserializer	GMSL3/2 → CSI-2

GMSL rates: GMSL1 ~3.125 Gb/s (NRZ, legacy); GMSL2 3 or 6 Gb/s fwd / 187.5 Mb/s rev (NRZ); GMSL3 3/6/12 Gb/s — 12 Gb/s is PAM4, 6 and below NRZ — backward-compatible GMSL3↔2↔1. Forward rates are fixed/selectable (CFG-pin resistors or register writes), not auto-negotiated like Ethernet. Typical AV topology: sensor → MAX9295/96717 (ser) → coax → MAX96712 (quad deser) → CSI-2 → SoC; on Jetson/Orin the deser feeds the SoC's CSI/VI block.

The non-obvious, test-relevant facts:

- The reverse/control channel is I2C tunneled over the same coax. The SoC configures the camera (exposure, gain, trigger, sync) through the deserializer, across the cable, to the serializer, to the sensor. A camera that “won't configure” can be a reverse-channel/ link problem, not a sensor problem. I2C address translation in the deserializer lets several identical sensors share one bus. This bidirectionality on one cable is GMSL's whole point.

- Power-over-Coax (PoC): the same coax carries power *and* GHz video; a PoC filter (ferrite/inductor + cap network) separates the DC power band from the signal band. A PoC filter issue breaks power *and* signal — and looks like “camera dead.” GMSL parts support line-fault detection to find PoC opens/shorts, which is a real test to read.
- Link lock is the GMSL equivalent of PCIe reaching L0. The deserializer has a per-link LOCK status bit (read over I2C or via the driver). No lock = no pixels. Per-device decode/line-CRC error counters accumulate on a marginal coax — trend them over a soak.
- The deserializer aggregates multiple cameras (e.g., MAX96712 = 4 links) and presents them to the SoC. Your test enumerates each link, not just “the deserializer.”
- FrameSync is GMSL’s multi-camera shutter synchronization: a master timer (in the deser or external) is tunneled over the reverse channel as a periodic GPIO that triggers all sensors together — essential for AV sensor fusion. Verify all cameras lock to the *same* FrameSync and that frame timestamps are coherent; FrameSync drift is a fusion bug that a single-camera capture test won’t see.

Test sequence (PCBA/module: often a known-good camera or fixture; vehicle: the real harness):

```
i2cdetect -y -r 1                                # find the deserializer (e.g., 0x29/0x48)
# read LOCK status (register/sysfs is driver/board specific):
cat /sys/bus/i2c/devices/1-0029/link_status      # "locked" / 1 == link up
v4l2-ctl --list-devices                          # enumerate capture devices the SoC sees
v4l2-ctl -d /dev/video0 --get-fmt-video          # resolution/format/framerate sane?
v4l2-ctl -d /dev/video0 --stream-mmap --stream-count=10 --stream-to=/tmp/f.raw # capture frames
cat /sys/bus/i2c/devices/1-0029/error_count      # non-zero == physical-layer errors
```

Capturing real frames is the end-to-end proof: sensor → serializer → cable → deserializer → CSI-2 → SoC all work. Vehicle phase is where GMSL marginality surfaces — the bench uses a short cable; the car uses 15 m of coax through a harness with real connectors and EMI. A link that locks on the bench and drops at temperature in the vehicle is the GMSL version of the PCIe “passes at 25 °C” escape, and the reason GMSL gets real coverage at EOL.

9.2 Automotive Ethernet — one pair, master/slave

Radar, lidar, and inter-module traffic ride automotive Ethernet: 100BASE-T1 / 1000BASE-T1 (and faster: 2.5/5/10GBASE-T1), a single twisted pair, full-duplex, PAM3-coded. Unlike consumer Ethernet, one PHY is the master (provides clock) and the other the slave — a mismatch means no link. PHYs (Marvell, Broadcom, TI) are configured over MDIO.

What you test and the tools:

```
ethtool eth1                                     # link up? speed? duplex? master/slave role?
ethtool -i eth1                                  # driver + firmware version
ethtool -t eth1 online                          # PHY/MAC self-test (offline test is more thorough but drops link)
ethtool -S eth1                                  # stats: rx/tx errors, CRC errors, dropped – your error counters
ethtool --cable-test eth1                      # TDR cable diagnostics (some PHYs): open/short + distance-to-fault!
iperf3 -c <partner> -t 30                      # throughput: 1000BASE-T1 should sustain ~0.95 Gbps
```

Real PHYs you’ll meet: Marvell 88Q2112 (100/1000BASE-T1), TI DP83TG721-Q1 (1000BASE-T1 with TSN/AVB + TC10 sleep/wake), Broadcom/Marvell multi-Gig families. They’re configured over MDIO (clause-22/45); Linux exposes them via the netdev + phylib.

Three senior notes:

1. Master/slave is a config, not a cable. Each automotive PHY link is set master (clock source) or slave, and the two ends must be *opposite*. A “no link” between two correctly-cabled PHYs is frequently a master/master or slave/slave misconfiguration, not a wiring fault — check the role (`ethtool eth1`) before you suspect the harness.
2. **ethtool --cable-test-tdr** uses TDR to report fault type + distance to fault — the killer automotive-Ethernet manufacturing tool, scope-free, the Ethernet cousin of PCIe lane margining. Result codes: OK,

Open Circuit (with distance), Short (to another pair), Impedance Mismatch (reflection from a discontinuity), Noise (test couldn't complete). On a harness the *distance* pinpoints the bad connector/segment. Pair it with `ethtool -S` (CRC/align/PHY-error counters) and an `iperf3` throughput run — good link + low `iperf` + rising CRC = marginal SI, confirmed by the TDR.

3. TSN / gPTP (IEEE 802.1AS, a profile of 1588 PTP) distributes a grandmaster clock for sensor fusion; the broader TSN suite adds time-aware shaping (802.1Qbv) and preemption. Validate with `ptp4l/phc2sys` and check offset-from-master convergence and `PHC↔system` clock sync. TC10 (OPEN Alliance) standardizes coordinated sleep/wake (local/remote wake, wake-forwarding, sleep negotiation) so the network can power down and a single event wakes it. You don't own gPTP/TC10 day one, but the words and the test hooks won't surprise you.

10 CAN / CAN-FD, For Test Engineers

CAN is a bonus qualification (“automotive protocols such as CAN”). The study guide §7 covers the protocol; here’s the test-engineering slice — the compute board has CAN transceivers you must prove work.

The bus in brief. Two wires (CANH/CANL), differential, multi-drop. Bits are “dominant” (0) or “recessive” (1); a dominant bit wins, which is how arbitration works — nodes start transmitting, the lower message ID wins the bus bit-by-bit without corrupting the winner (nondestructive). Classic CAN is ≤ 1 Mbps with an 8-byte payload; CAN-FD adds a faster data phase (multi-Mbps) and up to 64-byte payloads.

The two things that bite you in manufacturing:

1. Termination. The bus needs $120\ \Omega$ at each physical end. With both terminators present you measure $\sim 60\ \Omega$ across CANH/CANL with the bus powered off (two $120\ \Omega$ in parallel). Reading $120\ \Omega$ = one terminator missing; reading open or very low = wiring fault. A DMM across the bus is a 5-second test that catches the most common CAN problem.
2. Error counters / bus-off. Each node keeps a Transmit Error Counter (TEC) and Receive Error Counter (REC). As errors accumulate a node goes error-active $\rightarrow$ error-passive $\rightarrow$ bus-off (it removes itself from the bus). A board whose CAN goes bus-off under test has a transceiver, termination, or bit-timing problem. Watch these counters — they’re the CAN analog of AER/EDAC.

SocketCAN — Linux makes CAN a network interface:

```
ip link set can0 up type can bitrate 500000          # bring up at 500 kbps (classic)
ip link set can0 up type can bitrate 500000 dbitrte 2000000 fd on  # CAN-FD
ip -details -statistics link show can0              # state (ERROR-ACTIVE/BUS-OFF), TEC/REC, errors
candump can0                                         # watch all traffic
cansend can0 123#DEADBEEF                          # send a frame (ID 0x123)
cangen can0 -e -I 100 -L 8                         # generate traffic for a stress/loopback test
```

Manufacturing test of the transceiver: put the controller in loopback (internal, or an external loop back on the fixture), send known frames, verify they’re received intact, confirm the configured bitrate, and check that TEC/REC stay 0 and the interface stays ERROR-ACTIVE (never BUS-OFF). Add the DMM termination check and, for deeper coverage, a scope on CANH/CANL to verify the dominant/recessive voltage levels (~ 2.5 V common mode, ~ 2 V differential dominant). Higher layers — UDS (ISO 14229 diagnostics), DoIP, SOME/IP — sit on top and are awareness-level for you day one.

11 Instruments and Measurement

“Develop testing scripts and code to ... control test instruments, capture test parameters” is a core JD responsibility and one of your strongest existing skills (your `equipment_rpc.py` socket layer, your instrument drivers). This chapter is the bridge from what you did at X-ES to the Zoox bench.

11.1 The bench and what each instrument is for

Instrument	Measures / does	On a compute board you use it to...
DMM (digital multimeter)	V, I, R, continuity	Verify rail voltages, measure current draw, check CAN termination (Ω)
Oscilloscope	Time-domain waveforms	Power-up sequencing, ripple/noise, reset/PERST# timing, signal eye
Programmable power supply	Sources V/I	Power the DUT, sweep input voltage, test brown-out/UVLO
Electronic load	Sinks current	Load a rail/PSU to spec, test regulation and current limit
DAQ / switch matrix	Many channels	Scan many test points, thermocouples, route signals
Thermal forcer / chamber	Sets temperature	The hot/cold soak that surfaces marginal links
Signal/pattern generator, BERT	Sources signals	Stimulus for analog/serial paths

11.2 How you talk to them: VISA + SCPI

The stack hasn’t changed in 30 years and won’t surprise you:

- Transport: GPIB (IEEE-488, classic bench), USB, LAN/LXI (Ethernet — the modern default for a networked test station), RS-232.
- VISA (NI-VISA / pyvisa) abstracts the transport so the same code talks to a GPIB or LAN instrument by address string.
- SCPI (Standard Commands for Programmable Instruments) is the text command language most instruments speak: `*IDN?` (who are you), `*RST` (reset), `MEAS:VOLT:DC?` (measure DC volts), `SOUR:VOLT 12.0` (set output). Hierarchical, colon-separated, `?` = query.

```
import pyvisa
rm = pyvisa.ResourceManager()
dmm = rm.open_resource("TCPIP0::192.168.10.21::INSTR") # a LAN/LXI DMM
print(dmm.query("*IDN?")) # identify + prove the link works
dmm.write("CONF:VOLT:DC 10") # configure: DC volts, 10 V range
v = float(dmm.query("READ?")) # take a reading
psu = rm.open_resource("TCPIP0::192.168.10.30::INSTR")
psu.write("VOLT 12.0; CURR 5.0; OUTP ON") # source 12 V, 5 A limit, enable
```

Your `equipment_rpc.py` pattern — a socket server fronting shared instruments so several stations can use one expensive box — maps directly onto a Zoox bench. Bring it up as prior art: it’s exactly the “control test instruments” capability they’re asking for, already built.

11.3 Measurement discipline (where credibility is won or lost)

Reading a number is easy; reading a *trustworthy* number is the skill:

- 4-wire (Kelvin) for low resistance / current shunts. Lead resistance swamps a milliohm measurement; 4-wire removes it. Use it for shunts and rail-drop measurements.

- Settling and averaging. Let the rail settle after a load step before you read; average to beat noise — but know that averaging hides transients you may actually care about.
- Ranging. Fixed range for speed and repeatability in a test program; autorange for exploration. Range changes cost time and can glitch.
- Calibration and traceability. Instruments drift; they carry cal stickers with due dates and NIST-traceable calibration. A test result from an out-of-cal instrument is not a result. Your program should log instrument IDN + cal status with every measurement.
- GR&R. “Is my *measurement system* trustworthy?” Gauge Repeatability & Reproducibility (study guide §8.4) asks whether the same part measured repeatedly (by the same and different setups) gives consistent numbers. If your gauge variation is large relative to the tolerance, your pass/fail is noise. This is a real conversation you’ll have about any new measurement.

11.4 What you actually measure on a compute board

- Rail voltages under load against tolerance windows (often $\pm 3\text{--}5\%$): 12 V input, 5 V, 3.3 V, core VDD, DDR VDD/VPP, PHY supplies. Out-of-window under load = power-delivery problem (and a likely cause of PCIe/link errors).
- Current / power draw: idle and under stress; inrush at power-on; per-rail where shunts exist. Abnormal current = short, wrong stuffing, or a stuck device.
- Ripple and noise: scope, AC-coupled, bandwidth-limited, on each rail under load. Excess ripple correlates with link errors and is a finding even if the DC level is fine.
- Power sequencing: scope several rails on power-up — they must come up in the right order and timing, or devices can latch up or fail to train. A sequencing fault is a classic “trains intermittently / sometimes not detected” root cause.

The through-line: a lot of “digital” link failures are really power/SI failures. When PCIe throws correctable errors under load, the rail droop or ripple you catch with a scope is often the actual root cause. Being the test engineer who *reaches for the scope and the AER decode together* is how you close hard intermittent bugs.

11.5 The shared-instrument server pattern

A bench has expensive instruments (a good scope, a source-measure unit) that several stations want to share. Your `equipment_rpc.py` pattern — a socket server fronting shared instruments so several stations talk to one box by address — maps directly onto a Zoox bench and is exactly the “control test instruments” capability they’re asking for, already built. Bring it up as prior art. The design lesson worth stating: an instrument abstraction should mirror your DUT-backend abstraction — a real `pyvisa`/SCPI backend *and* a mock instrument for laptop demos and unit tests — so the instrument-driving code is testable against canned responses the same way the PCIe code is testable against a mock backend.

11.6 Reliability & stress screening (where thermal/vibration belong)

“Passes at 25 °C, fails at 85 °C” is the defining manufacturing-test reality, and it’s why the thermal forcer/chamber is on the bench. Two physics reasons to be able to state:

1. High-speed links lose margin with temperature. Conductor loss and jitter rise with temperature, so a SerDes link (PCIe, GMSL, automotive Ethernet) that equalizes to an open eye at 25 °C can have a *closed* eye — link errors, retrains, or a speed/width fallback — at 55–85 °C. This is why lane margining and AER monitoring must be done hot, not just at ambient.
2. Failure rate is Arrhenius in temperature. Temperature-activated failure mechanisms follow an exponential law; the rule of thumb is failure rate roughly doubles per ~ 10 °C. Elevated-temperature life tests are processed *through* the Arrhenius equation to predict normal-temperature behavior — the theory under burn-in/HASS.

The stress techniques and where each belongs (the split: HALT is a design tool; HASS/ESS/burn-in are production screens):

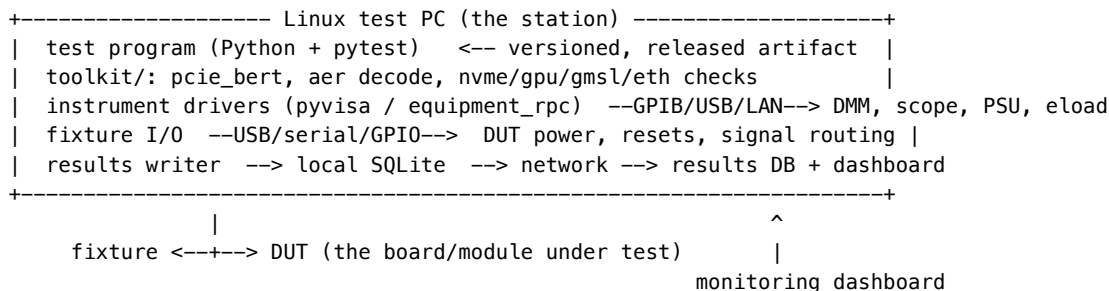
Technique	Stress	Applied to	Where in flow
HALT (Highly Accelerated Life Test)	Temp + multi-axis vibration <i>beyond</i> spec, to destruction	Prototypes (DV)	NPI / reliability — finds the design's limits and <i>derives the HASS profile</i>
HASS (Highly Accelerated Stress Screen)	HALT-derived profile, near/just beyond operating limits	Production units	Production screen (post-assembly)
ESS (Environmental Stress Screening)	Thermal cycling + vibration <i>within</i> spec	Production units	Production screen — infant-mortality/workmanship escapes
Burn-in	Steady elevated temp, powered/under load, hours	Production units	Module/system — screens infant mortality
Thermal cycling	Repeated hot↔cold ramps	Both	DV reliability + production ESS — solder-fatigue/CTE-mismatch

Your contribution to all of these is the in-soak functional monitor. The screen *precipitates* the latent defect (the oven/shaker provides the stress); *your* test provides the at-temperature link/error/throttle checks — margin the lanes hot, watch AER/ECC/EDAC/ SMART/XID deltas vs temperature — that turn “we baked it” into “we baked it and proved every interface still trains clean hot.” A thermal test that never actually gets the part hot, or a soak with no functional monitor running during it, is a test that can't catch the defect it exists for.

12 The Test Station and the Test Framework

The JD’s verbs — *build and release test solutions, develop scripts to verify functionality, capture test parameters, analyze results for continuous improvement* — describe a test framework, not a pile of scripts. You’ve built one before (pytest fixtures, the FastAPI station dashboard, `equipment_rpc.py`). This chapter is how that maps to Zoox and what the companion toolkit/ implements.

12.1 Anatomy of a station



Many identical stations run the SAME test-program version, report to the SAME dashboard.

The “many identical stations, one dashboard” picture is exactly your X-ES/2G setup — the heartbeat dashboard you built on your own initiative is the proof you already think this way.

12.2 Test program architecture

- pytest as the runner. Each test is a function with a clear pass/fail; @parameterize sweeps it over every DUT interface (every BDF, every NVMe, every GPU) from a config; fixtures handle setup/teardown (power the DUT, load firmware, arm counters) and guarantee cleanup even on failure. This is the structure the toolkit's harness uses.
- Data-driven via a topology/expectation config. Don't hard-code "GPU at 03:00.0 should be Gen4 x16." Put the *expected topology* in a YAML/JSON file (which devices, expected speed/width, which NVMe models, rail limits) and let one generic test assert reality against it. New board revision → new config, not new code. The toolkit ships exactly this file format.
- Capture parameters, not just verdicts. Log every *measured value* (link speed, margin, CE count, rail voltage, temperature, throughput) alongside pass/fail. A test that only records PASS/FAIL throws away the data you need for SPC and limit-setting. This is the literal meaning of "capture test parameters ... for continuous improvement."
- Structured results + artifacts. Each run emits a structured record (station, DUT serial, test-program version, per-test results with values+limits, timestamps, operator) to SQLite locally and a central DB. Failures attach their evidence (decoded AER, dmesg snippet, the eq/margin matrix) so the next engineer doesn't re-reproduce to diagnose.
- Operator UX. Big clear PASS/FAIL, serial-number/barcode scan to start, no ambiguous states, and recoverability (your "restart from any keyword" feature is the kind of operator-centered design that matters on a line). The operator is your user; design for them.

12.3 Releasing and versioning test programs

This is the “build and release test solutions ... at Zoox and at contract manufacturing partners” responsibility, and it’s where a senior role differs from writing scripts:

- A test program is a released artifact, versioned and tagged like firmware. A given unit's record must say *which test-program version* tested it. When you change limits or add coverage, that's a new release with notes.
- Config separate from code. Limits, topology, and station-specific settings live in config so the same code runs at Zoox and at a CM with different fixtures.

- Golden / known-good units. Keep characterized reference units. Before/after a release, run them to prove the test still passes good boards and fails known-bad ones — and to correlate your station against a CM's station (same unit, same result, both sites).
- CM release process. You package the program, document setup, train remotely, and support failures you can't physically touch. (Guide B covers the working relationship.) Robustness and clear logs matter ten times more when you can't walk over to the station.

13 Test Economics: Deployment, Runtime, and Yield

The JD names your continuous-improvement mandate precisely: “*drive continuous improvement for improving test deployment, test runtime, and yield.*” These are the three numbers a test engineer is judged on. Know what each means and its levers.

13.1 Yield

Yield = fraction of units that pass. The decomposition you must know:

- FPY (First Pass Yield) — fraction passing the first time, no retest/rework.
- RTY (Rolled Throughput Yield) — product of FPY across all steps. Five steps each at 98% $\rightarrow 0.98^5 \approx 90\%$. This is why every added test step costs yield, and why you don’t add coverage casually.

The two ways a test program hurts yield wrongly: false fails (good units failing — gauge noise, too-tight limits, flaky tests) and escapes (bad units passing — too-loose limits, missing coverage). You’re constantly trading these off. The data discipline below is how you set limits so you minimize both.

- SPC (Statistical Process Control) — plot measured parameters over time on control charts; the process talks to you through drift and outliers *before* it makes scrap.
- Cpk (process capability) — how well the measured distribution fits inside the spec limits ($Cpk \geq 1.33$ is the usual “capable” bar). Low Cpk = the process and the limits are too close; you’ll bleed yield no matter how good the test is.
- Data-driven limits. Set pass/fail limits from the measured fleet distribution (e.g., mean $\pm$ a margin, guard-banded for gauge error), not from a guess. This is why “capture parameters” matters: you can’t set a good limit on data you didn’t keep.
- Pareto + RCA. 80% of fails come from ~20% of causes. Pareto the failure modes, attack the top bar, use 5-whys / fishbone to get to root cause, verify the fix moved the bar.

13.2 Runtime

Test time is money — it sets line throughput and station count. Levers:

- Parallelize. Run independent tests concurrently (NVMe fio soak *while* GPU gpu-burn *while* the PCIe monitor watches AER). pytest-xdist / async / threads. Your station should be busy on many things, not serial.
- Right-size soaks. This is where your BERT confidence target is an *economic* tool: run exactly long enough to prove 1e-12 at 95% confidence and stop — not a fixed, conservative, padded duration. Choosing CL instead of a wall-clock time is a runtime optimization with a statistical guarantee.
- Move tests left. A test that runs faster/cheaper at PCBA than at module test should run at PCBA. (Balanced against the 10 $\times$ escape cost — don’t move a test earlier than the phase that can actually catch its defect.)
- Cut tests that never catch anything. If the data shows a test has caught zero real defects across 10,000 units and costs 30 s, that’s a candidate to drop or sample. The fleet data tells you which tests earn their runtime.

13.3 Deployment

How fast and how reliably a new/updated test reaches every station and CM. Levers: config over code (no code change to retarget), versioned releases with rollback, golden-unit correlation gating a rollout, remote update of stations, and clear release notes. A test that takes a week to deploy to a CM is a worse test than a slightly less thorough one that deploys in an hour — *deployment is a first-class metric*, which is why the JD names it.

14 Mass-Production Realities

The JD’s “manufacturing lines at Zoox and at contract manufacturing partners” and the “deployment / runtime / yield” triad live at volume — and volume has realities that never show up on a single bench. This chapter is the things that dominate at scale, and the statistical/data discipline that turns captured parameters into limits and yield.

14.1 Takt, cycle time, and parallel / multi-up test

- Takt time = available production time ÷ required output — the drumbeat the line must hit. Your test’s cycle time must fit inside takt or the station becomes the bottleneck (and you need more stations = more capital).
- Parallel / multi-up / multisite test is the primary runtime lever: test N DUTs at once on one station (multisite), and run independent tests concurrently on one DUT (NVMe `fio` soak || GPU stress || PCIe AER monitor). This is how you amortize a fixed soak across throughput. The other big lever is right-sizing soaks — the BERT *confidence target* instead of a padded fixed time, run exactly long enough to prove the target and stop.
- Scale intuition: a station producing one record per cycle at a 30 s takt generates ~100,000 records/month. That’s well within ordinary database capacity, but it tells you the data-volume scale you’re designing the results store for.

14.2 Test-station architecture and the test executive

A station is a Linux test PC running a *versioned, released* test program (pytest + toolkit), instrument drivers (VISA/SCPI over LAN/USB/GPIB), fixture I/O (DUT power, resets, signal routing), and a results writer that syncs to a central DB + dashboard. Many identical stations run the same program version and report to the same dashboard. Off-the-shelf test executives (NI TestStand, Keysight OpenTAP/PathWave) provide sequencing, per-step limits, result logging, and operator UI — the build-vs-buy decision is real, and a pytest-based harness is a valid “build” answer for a Linux/Python shop (which is what the toolkit is).

14.3 Traceability & genealogy

Every unit carries a serial number (1-D/2-D barcode, Direct Part Marking, or RFID), scanned to *start* test, so each result row records which unit, which station, which program version, when. Genealogy records which component lots / sub-assembly serials went into each finished unit — so when a bad lot or a failing test mode is found, you can trace every affected unit quickly (top-down and bottom-up). Barcode entry also removes keystroke errors. For Zoox compute this is also a *quality* lever — it’s how logging `power_on_hours` / `data_units_written` catches re-labeled NVMe stock even when the unit passes.

14.4 Golden units & cross-site correlation

- Keep golden units: characterized known-good *and* known-bad references.
- Before/after every release, prove the test still passes the known-good and fails the known-bad — this catches a too-strict (false-fail) or too-loose (escape) change *before* the line, not on it.
- Cross-station / cross-site correlation: run the *same* golden unit at Zoox and at the CM; the stations must agree. A correlation gap is a fixture/calibration/environment difference you must resolve before trusting their yield. Operationally this is a Gauge R&R reproducibility study across sites.

14.5 FAI, CM workflows, MES/OEE

- FAI (First Article Inspection): the first unit(s) off a new line/process/revision get a thorough, documented verification before volume is released — the production-side gate that the line is set up correctly.
- CM workflow: Zoox releases test programs to EMS partners (Flex/Jabil-type) who build at volume on units you may never touch. The release package is the artifact: versioned code

- config, setup/runbook, fixture spec, acceptance criteria, golden-unit correlation data, triage guide. (Guide B has the full working relationship.)
- MES (Manufacturing Execution System) owns work-order release, electronic work instructions, serialization, genealogy/traceability, quality/NCR handling, and OEE (Overall Equipment Effectiveness = Availability $\times$ Performance $\times$ Quality). Your station typically checks in/out with MES (is this serial allowed to test here? record the verdict back) and streams parameters to a test-data-management/analytics layer. A test station drags OEE down through downtime (Availability), slow cycles (Performance), and false-fails/retests (Quality).

14.6 The statistics that turn parameters into limits

This is the quantitative core of “analyze results for continuous improvement,” and where DV-captured distributions become MT limits:

- SPC (control charts). Plot each captured parameter over time with $\pm 1/2/3\sigma$ zones; the Western Electric rules (1 point beyond 3σ ; 2 of 3 beyond 2σ same side; 4 of 5 beyond 1σ ; 8 in a row one side) flag drift *before* it makes scrap. A process can be in control yet not capable — which is why control charts pair with capability. For per-unit test data, I-MR (individuals / moving-range) charts are the natural fit.
- Capability: C_{pk} / P_{pk} . C_p = spec width $\div$ process spread; C_{pk} accounts for *centering*. P_p / P_{pk} are the short-term/preliminary analogs used in DVT/PVT *before* the process is proven stable; C_p / C_{pk} once control charts show stability. Bars: $C_{pk} \geq 1.33$ = “capable,” ≥ 2.0 = world-class. A low C_{pk} means the process and the limits are too close — you’ll bleed yield no matter how good the *test* is, which is itself a finding for EE/process.
- MSA / Gauge R&R. Before trusting a limit, prove the *measurement system*. Gauge R&R (Repeatability — same setup repeated; Reproducibility — across operators/stations) quantifies how much observed variation is the gauge vs the part. AIAG study: 10 parts $\times$ 3 operators $\times$ 3 repeats. Acceptance: %GRR < 10% acceptable, 10–30% conditional, > 30% unacceptable; also $ndc > 5$. If gauge variation is large vs the tolerance, your pass/fail is *noise* — and cross-CM correlation is exactly a reproducibility study.
- Guard-banding + data-driven limits. A guard band tightens the *test* limit inside the *spec* limit by an amount tied to measurement uncertainty (from the GR&R), so gauge error doesn’t pass a truly-bad unit. The senior workflow: (1) capture the parameter on many units across corners (DV); (2) build the distribution; (3) set the MT limit from the distribution + a guard band — *not* a guess (“fleet margin is 0.42 ± 0.04 UI; a 0.25 UI limit gives $C_{pk} \approx 1.4$ with room for gauge error”); (4) monitor with SPC and re-tune when the process moves.

14.7 False-fail vs escape economics — and why it’s not symmetric here

The false-fail vs escape trade is normally an economic decision — but in a robotaxi it’s asymmetric. A false fail costs throughput, retest labor, and (at a CM) remote firefighting. An escape costs the 10 $\times$ -per-phase curve and, for safety-critical compute, a field/safety event whose cost is effectively unbounded. So the standing rule is “ship only good units”: you do not buy yield by loosening a limit that lets a real defect through. You buy yield by *reducing false fails* — better limits via data + guard-banding, less gauge noise, less test flakiness — never by raising the escape rate.

ISO 26262 awareness (it’s a safety product). You’re not the functional-safety owner, but a robotaxi compute role assumes baseline awareness: ASIL levels (A $\rightarrow$ D), higher ASIL $\Rightarrow$ higher required coverage, mandatory traceability, documentation as a safety case. Concretely that means: the serial $\rightarrow$ genealogy $\rightarrow$ program version $\rightarrow$ measured results $\rightarrow$ disposition chain must be auditable; which defect each test catches and why a limit sits where it does are safety-case evidence (so the data-driven-limit discipline is also a safety input); and a test-program or limit change touches the safety argument — another reason for versioned, gated, documented releases. Higher-layer diagnostics you’ll meet at the edges: UDS (ISO 14229), DoIP, SOME/IP — awareness-level day one.

15 The Linux Command-Line Debugging Toolbox

“Experience with the Linux platform — debugging failures using command line tools” is a hard requirement. You know Linux; this is the hardware-debugging subset as a fast reference, grouped by what you’re chasing.

Enumerate / “is it even there?”

```
lspci -nn ; lspci -vvv -s <bdf> ; lspci -t      # PCIe: devices, detail, topology tree
lspci -k -s <bdf>                               # which kernel driver is bound
nvme list -o json ; nvidia-smi -L              # NVMe / GPU enumeration
ip -details link show ; ethtool <iface>        # network interfaces incl. CAN/auto-Eth
i2cdetect -y -r <bus> ; i2cget/i2cset          # I2C devices (GMSL deser, sensors, PMICs)
lsusb ; lsblk ; dmidecode -t memory            # USB, block devices, DIMM inventory
```

Kernel’s side of the story

```
dmesg -T --level=err,warn ; dmesg -w           # kernel log (timestamped; -w = follow)
dmesg | grep -iE 'pcie|aer|nvme|xid|link|train' # the words that matter for link failures
journalctl -k -b ; journalctl -u <service> -f  # kernel log this boot; follow a service
```

Drivers, modules, interrupts

```
lsmod ; modinfo <mod> ; modprobe <mod> ; rmmod # loaded modules; info; load; unload
cat /proc/interrupts | grep <dev>               # is the device actually interrupting?
ls /sys/bus/pci/drivers/<drv>/ ; echo <bdf> > .../unbind # rebind a device to a driver
```

Counters / health (the “arm → stress → read” sources)

```
setpci -s <bdf> ECAP_AER+0x10.L                # read/clear AER (toolkit does this cleanly)
nvme smart-log /dev/nvme0 -o json              # NVMe health
nvidia-smi -q -d ECC,PCIE,PERFORMANCE,TEMPERATURE # GPU ECC/replay/throttle/temp
ras-mc-ctl --errors ; cat /sys/devices/system/edac/mc/mc*/ce_count # memory ECC
ethtool -S <iface>                             # NIC/PHY error stats
sensors ; cat /sys/class/thermal/thermal_zone*/temp # board/CPU temperatures
```

Performance / what’s the bottleneck

```
top / htop ; uptime                           # load, what's hot
iostat -x 1 ; mpstat -P ALL 1 ; sar            # disk / per-CPU / historical
perf top ; perf stat <cmd>                    # where cycles go
```

The meta-skill isn’t memorizing flags — it’s the reflex: failure → enumerate (is it there?) → dmesg (what did the kernel see?) → counters (arm/stress/read) → isolate (swap/reseat, known-good) → measure (scope/DMM) → decode to a physical part. Guide B turns that reflex into an explicit method.

16 Your First 90 Days — Technical Ramp

(Guide B covers the *career/relationship* 30-60-90. This is the *technical* ramp — what to learn and do with the hardware.)

Week 1 — orient and observe.

- Get a station and the current Linux test image; get accounts/access (results DB, dashboard, source control, the issue tracker).
- Run the *existing* test program on a known-good unit end to end. Read its code. Map it to the four phases. Find where results land.
- Build the platform's topology in your head: pull `lspci -t`, `nvme list`, `nvidia-smi topo -m` on a real unit; sketch the root ports, switches, retimers, GPUs, NVMe, GMSL deser, NICs, CAN. *Don't change anything* — read-only, learn the normal.
- Run the toolkit you brought, in mock mode, and show one person. (See Guide B on how to introduce it without overstepping.)

Month 1 — own a corner.

- Take ownership of one interface's test (likely PCIe — your strength). Understand its current coverage, limits, and failure history from the data.
- Reproduce a known failure mode deliberately and watch every counter, so you trust the test.
- Make one small, safe improvement: better failure logging, an AER decode, a clearer operator message. Land it through the team's release process. First contribution = trust.

Months 2-3 — improve and extend.

- Pick a real continuous-improvement target from the data: a top Pareto failure mode, a slow test step, or a coverage gap (e.g., “we pass on link-up but don't measure per-lane margin”). Propose it with data, build it, correlate on golden units, release it.
- Support a new-board bring-up or a new HW generation with EE — the JD's “support test and validation of prototype designs.” This is where your bring-up checklist (PCIe chapter) earns its keep.
- Be the person who, when a board fails intermittently under thermal, reaches for the AER decode *and* the scope and closes it.

What “ramped” looks like at 90 days: you can take a new board from EE, stand up coverage across the right phases, set data-driven limits, deploy it to a line or CM, and debug a hardware failure to a physical root cause — independently. That's the JD, delivered.